

Hiwonder 幻尔科技

语音交互应用



Hiwonder 官方网站



目录

1. 六路麦克风阵列配置工作（必读！）	6
1.1. 申请离线语音资源包和 ID	6
1.2. 替换离线语音资源和 ID	10
2. 语音控制小车移动	13
2.1. 实验简介	14
2.2. 准备工作	14
2.3. 实验步骤	14
2.4. 实现效果	15
2.5. 程序分析	16
2.5.1. launch 启动文件	16
2.5.2. Python 程序	17
2.6. 功能延伸	19
2.6.1. 修改唤醒词	19
3. 语音控制机械臂	20
3.1. 实验简介	20
3.2. 准备工作	20
3.3. 实验步骤	21
3.4. 程序分析	22
3.4.1. launch 启动文件	22
3.4.2. Python 启动文件	23
4. 语音控制颜色识别	24
4.1. 实验简介	24

4.2. 准备工作	25
4.3. 实验步骤	25
4.4. 实现效果	26
4.5. 程序分析	27
4.5.1. launch 启动文件	27
4.5.2. Python 程序	28
5. 语音控制颜色追踪	31
5.1. 实验简介	31
5.2. 准备工作	31
5.3. 实验步骤	32
5.4. 实现效果	33
5.5. 程序分析	34
5.5.1. launch 启动文件	34
5.5.2. Python 程序	35
6. 语音控制色块分拣	36
6.1. 实验简介	36
6.2. 准备工作	37
6.3. 实验步骤	37
6.4. 功能实现	38
6.5. 程序分析	38
6.5.1. launch 启动文件	38
6.5.2. Python 程序	39

7. 语音控制垃圾分拣	40
7.1. 实验简介	40
7.2. 准备工作	40
7.3. 实验步骤	41
7.4. 功能实现	41
7.5. 程序分析	42
7.5.1. launch 启动文件	42
7.5.2. Python 程序	43
8. 语音控制多点导航	43
8.1. 实验简介	44
8.2. 准备工作	44
8.3. 实验步骤	44
8.4. 功能实现	45
8.5. 程序分析	46
8.5.1. launch 启动文件	46
8.5.2. Python 程序	47
8.6. 功能拓展	48
9. 语音控制导航搬运	49
9.1. 实验简介	50
9.2. 准备工作	50
9.3. 实验步骤	50
9.4. 功能实现	51

9.5. 程序分析	52
9.5.1. launch 启动文件	52
9.5.2. Python 程序	53

1. 六路麦克风阵列配置工作（必读！）

1.1. 申请离线语音资源包和 ID

由于我们使用的是离线语音识别，所以需要申请科大讯飞的离线语音资源包。这里需要注意，目前科大讯飞离线语音资源包只支持国内注册用户。

1) 复制网址：<https://www.xfyun.cn/>，打开科大讯飞开放平台网页，点击注册账号。



2) 选择 **手机号登录**，填写相对应的信息。（国外用户可选择所对应国家的区号）

微信扫码

手机快捷登录

账号密码登录



+86



请输入手机号

请输入手机验证码

[获取验证码](#)

☐ 未注册的手机号将自动注册。勾选即代表您同意并接受[服务协议](#)与[隐私政策](#)

登录

3) 注册完成后, 点击 **控制台->我的应用** , 新建应用。



3) 根据提示信息输入完内容后, 点击 **提交** 。

为保证账户安全，平台将对未实名认证用户进行能力调用次数以及其他权益限制，建议您优先完成实名认证 [立即实名认证 >>](#)

我的应用 > 创建应用

* 应用名称

请设定一个应用名称，少于30个字符

请填写应用名称

* 应用分类

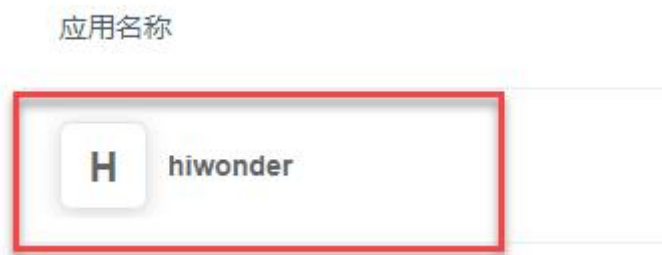
应用分类标签最多选一个

* 应用功能描述

请简述使用场景、应用特点等信息，不超过300个字符

提交 返回我的应用

4) 打开刚刚新建的应用。



5) 点击 **离线命令词识别**，可以在下图红框内找对应的 APPID，然后找到 **离线命令词识别 SDK->Linux MSC**，点击下载。

hiwonder

语音识别

语音识别 (流式版)

语音识别

极速语音识别

实时语音识别

离线语音识别

语音唤醒

语音唤醒 API

离线命令词识别

实时用量

今日实时装机量

0

剩余装机量

10

有效期至 2022-10-19

立即购买

服务接口认证信息

APPID 21 31

离线命令词识别 SDK

SDK名称	版本	操作
Android MSC	1143	下载 文档
Linux MSC	1227	下载 文档
Windows MSC	1126	下载 文档
iOS MSC	1174	下载 文档

*讯飞语音SDK也支持多种能力的打包组合，若有需要请前往组合SDK页面下载组合SDK。

6) 点击返回旧版。

SDK下载中心

欢迎使用讯飞开放平台开发者工具套件 (SDK)，SDK提供多种语言版本，统一了接口调用方式。

[返回旧版](#)

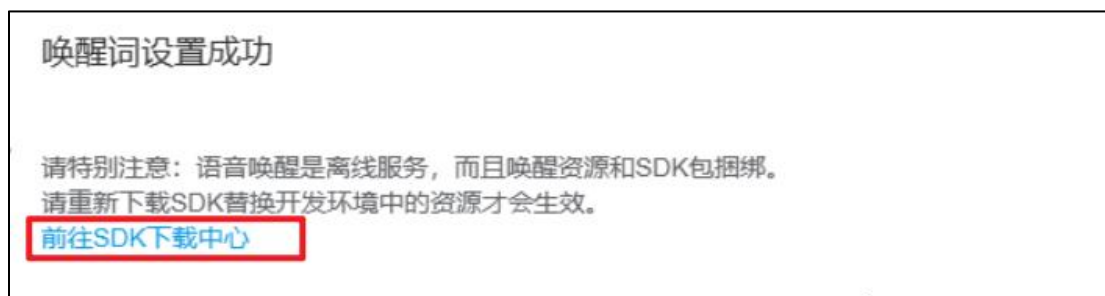
7) 选择“Linux”，选中需要的功能，点击“SDK 下载”，再点击“确定”开始下载。



8) 可以点击“个性化唤醒词体验包”，可以设置唤醒词为“小迈小迈”，然后点击“提交”。



9) 点击“前往 SDK 下载中心”，然后重复第 7) 的步骤，开始下载。



注意：每次申请一个新的应用只能免费使用九十天，后续使用需要收费。当我们的应用到期的时，我们可以申请新的应用，每个人可以申请五次。申请新应用的方法，与上文一致。

1.2. 替换离线语音资源和 ID

1) 将下载好的语音 SDK 压缩包文件解压。

 Linux_aitalk_exp1227_216da28f.zip 2022/7/21 星期四 15:... WinRAR ZIP 压缩... 12,441 KB

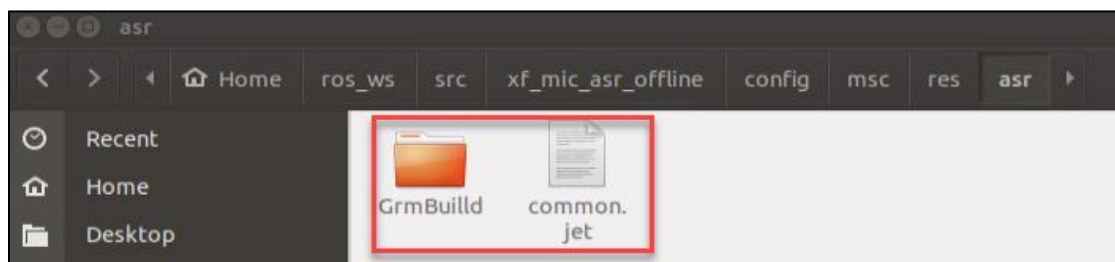
注意：这里需要根据自己设置的下载路径找到相关文件。

2) 找到解压好的文件夹“Linux_aitalk_exp1227_216da28f”（1227_216da28f 为版本 ID，并非固定，会根据官网来决定），进入“\bin\msc\res\asr”文件夹，找到“common.jet”。将文件拖动到镜像桌面内。

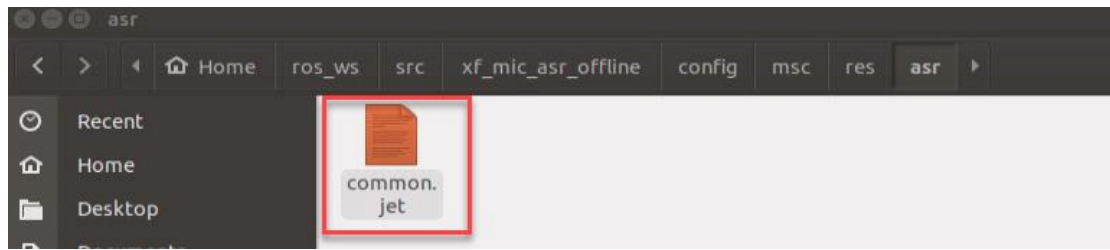


3) 打开虚拟机内的 Ubuntu 系统，打开路径：

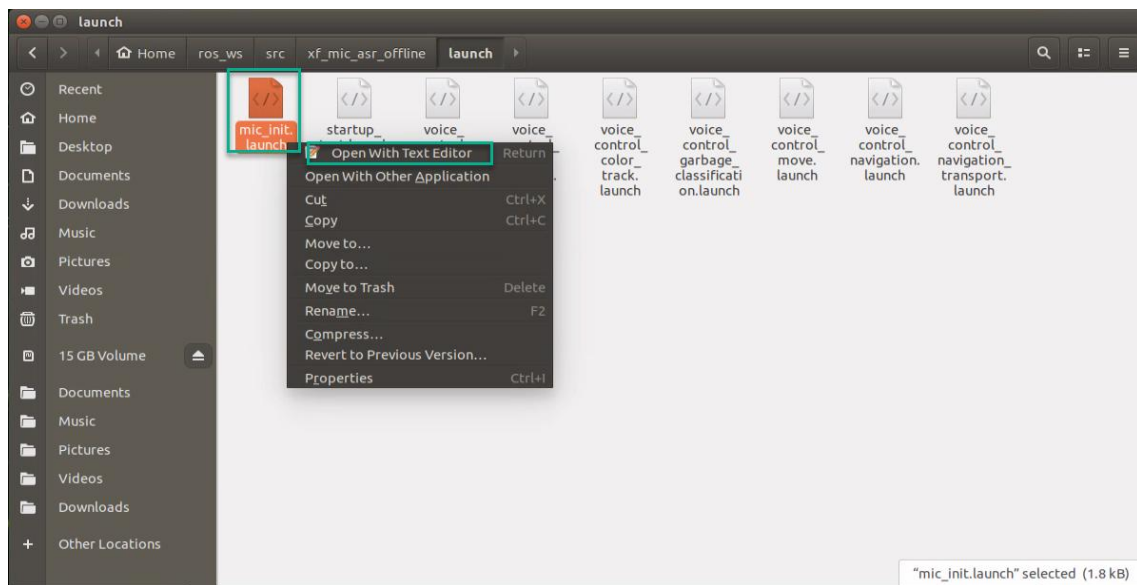
“Home/ros_ws/src/xf_mic_asr_offline/config/msc/res/asr/”，进入存放离线语音资源的文件夹，将红框内的文件删除。



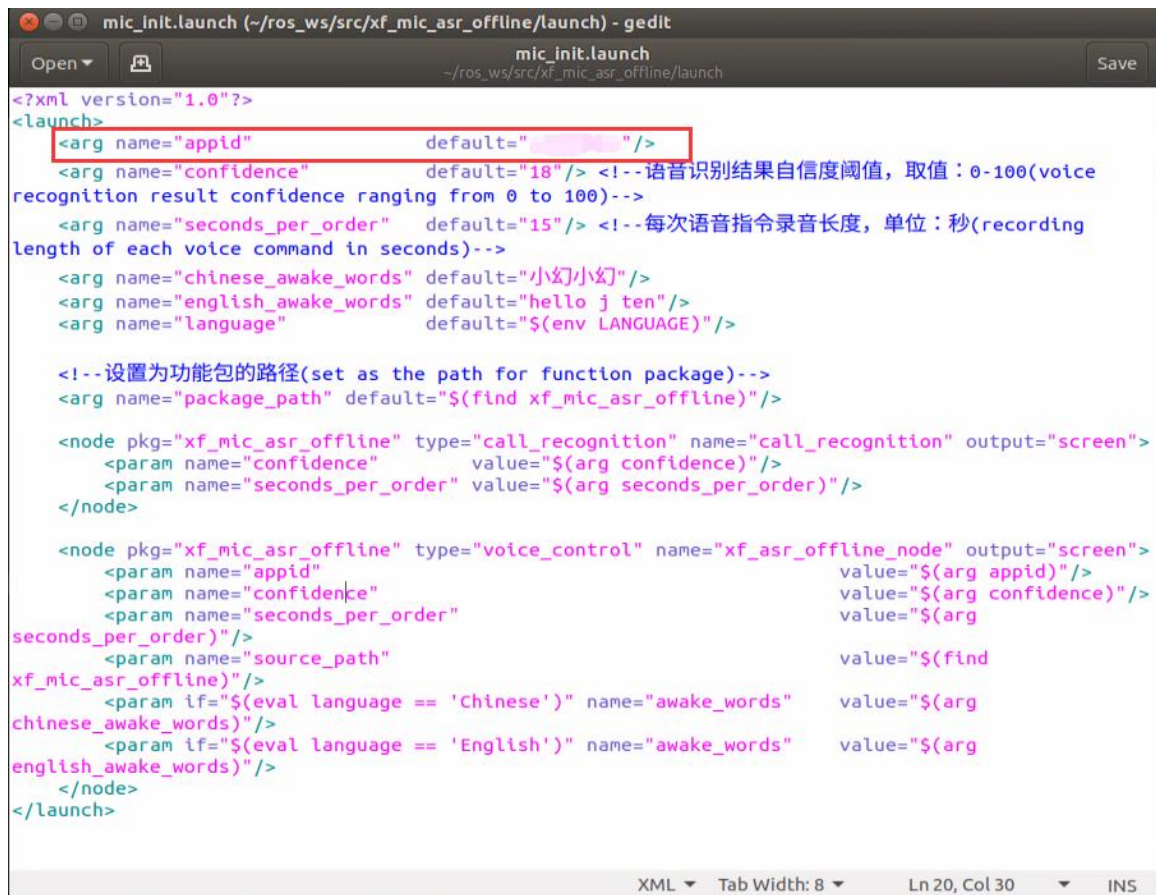
4) 将“common.jet”获得的文件放入文件夹。



5) 打开路径“Home/ros_ws/src/xf_mic_asr_offline/”, 右键“mic_init.launch”文件，选择绿框内容。



6) 将红框内的 appid 设置成在科大讯飞得到的 appid，然后按下“Ctrl+S”保存。



```
<?xml version="1.0"?>
<launch>
  <arg name="appid" default="" />
  <arg name="confidence" default="18"/> <!-- 语音识别结果自信度阈值, 取值: 0-100(voice
recognition result confidence ranging from 0 to 100)-->
  <arg name="seconds_per_order" default="15"/> <!-- 每次语音指令录音长度, 单位: 秒(recording
length of each voice command in seconds)-->
  <arg name="chinese_awake_words" default="小幻小幻"/>
  <arg name="english_awake_words" default="hello j ten"/>
  <arg name="language" default="$(env LANGUAGE)"/>

  <!-- 设置为功能包的路径(set as the path for function package)-->
  <arg name="package_path" default="$(find xf_mic_asr_offline)"/>

  <node pkg="xf_mic_asr_offline" type="call_recognition" name="call_recognition" output="screen">
    <param name="confidence" value="$(arg confidence)"/>
    <param name="seconds_per_order" value="$(arg seconds_per_order)"/>
  </node>

  <node pkg="xf_mic_asr_offline" type="voice_control" name="xf_asr_offline_node" output="screen">
    <param name="appid" value="$(arg appid)"/>
    <param name="confidence" value="$(arg confidence)"/>
    <param name="seconds_per_order" value="$(arg
seconds_per_order)"/>
    <param name="source_path" value="$(find
xf_mic_asr_offline)"/>
    <param if="$(eval language == 'Chinese')" name="awake_words" value="$(arg
chinese_awake_words)"/>
    <param if="$(eval language == 'English')" name="awake_words" value="$(arg
english_awake_words)"/>
  </node>
</launch>
```

7) 如果是国外用户的话, 在修改完 APPID 以及替换 common.jet 文件后。还需要连接



机器人进入到远程桌面中, 双击打开桌面的“语音功能”这一项, 将其修改为“English”, 这样后续的语音玩法才能切换为英文。



2. 语音控制小车移动

本节课是对机器人发布对应的语音指令来控制机器人做出对应的移动, 例如语音控制小车前进、后退等。

2.1. 实验简介

首先，订阅麦克风阵列节点发布的语音识别服务，对语音进行定位、降噪、识别等操作后，获取识别的语句和声源的角度，以及打开机器人的底盘移动控制服务，方便后续发布语音指令直接执行。

随后，用特定唤醒词唤醒机器人。说出唤醒词后，机器人会进行相应的语音反馈回答。

最后，进行语句匹配，根据匹配结果，机器人会执行对应的动作。

2.2. 准备工作

1) 在本节课开始前，需要将语音模块安装至机器人上（如已安装可跳过），并连接至集线器的 USB 接口；

2) 请参考本目录下的“[1.六路麦克风阵列配置工作（必读必做！）](#)”参看 Jetson Nano 部分完成对 APPID 的申请和对文件的替换。

3) 系统出厂默认使用英文的唤醒词“hello hiwonder”进行唤醒，如果需要更换语言为中文或使用的是“AI 语音盒子（WonderEchoPro）”需要烧录语音交互应用指令词，参考同目录下“\语音交互应用\中英文唤醒词切换”资料进行语言切换或指令词烧录。

2.3. 实验步骤

注意：输入指令时需要严格区分大小写，且可使用“Tab”键补齐关键词。

1) 启动机器人，将其连接至远程控制软件 NoMachine。关于如何连接可参考“1 教程资料\1 机器人快速使用指南\配套手册\1.JetRover 使用入门->4.开发环境搭建”。

2) 点击系统桌面的图标，打开命令行终端。

3) 输入指令，并按下回车，关闭手机 APP 服务。

```
sudo systemctl stop start_app_node.service
```

```
hiwonder@hiwonder:~$ sudo systemctl stop start_app_node.service
```

4) 输入指令，并按下回车，开启语音控制小车玩法。

```
roslaunch xf_mic_asr_offline voice_control_move.launch
```

```
hiwonder@hiwonder:~$ roslaunch xf_mic_asr_offline voice_control_move.launch
```

5) 如需关闭此玩法，可在终端界面按下“Ctrl+C”。若关闭失败，请反复尝试。

6) 若后续想要重新体验有关手机 APP 的玩法，则需要输入指令重启相关服务。输入并回车执行指令后，等待机械臂恢复至初始姿态即可。

```
sudo systemctl start start_app_node.service
```

```
hiwonder@hiwonder:~$ sudo systemctl start start_app_node.service
```

2.4. 实现效果

注意：

1. 为保证良好的体验效果，请在相对安静的环境下进行。
2. 在每次说语音指令前，建议先说唤醒词。
3. 说语音指令时声音要洪亮、清晰。
4. 语音指令要一条一条进行，在机器人执行完一次反馈后再说出其它指令。

1) 启动玩法后，等待扬声器播放“准备就绪”后，即可开始语音输入。

2) 我们需要先说唤醒词“hello hiwonder”，等待扬声器播放“我在”，再发出下个语音指令。例如说出“前进”，当机器人识别到语音指令后，扬声器会播放“好的，开始前进”，然后机器人即可进行指令对应的移动。

指令语句及其对应的控制动作如下：

指令语句	对应功能
前进	控制小车前进
后退	控制小车后退

左转	控制小车左转弯
右转	控制小车右转弯
过来	控制小车移动到发出指令者的前方

2.5. 程序分析

语音控制机器人移动，是让语音控制节点跟机器人底层驱动节点建立联系，在通过声音发出的命令来控制小车按命令执行对应动作。

2.5.1. launch 启动文件

该 launch 启动文件路径位于：

```
/home/hiwonder/ros_ws/src/xf_mic_asr_offline/launch/voice_control_move.launch
```

◆ 启动多个 launch 文件

<include>标签在这里用于包含其他的 launch 文件，在该文件被启动时，会将其他的 launch 文件一同启动。

这里主要是启动了麦克风的初始化、底盘控制、雷达等启动文件。

```
<include file="$(find xf_mic_asr_offline)/launch/mic_init.launch"/>
<include file="$(find hiwonder_controller)/launch/hiwonder_controller.launch"/>
<include file="$(find hiwonder_peripherals)/launch/lidar.launch"/>
<include file="$(find hiwonder_servo_controllers)/launch/start.launch"/>
```

◆ 启动节点

<node>为文件中启动的节点，后续的<name>、<pkg>、<type>、<output>等都为该节点内的参数。

<name>是该节点名字，<pkg>就是该节点所在的功能包；

<type>是该节点执行的文件名称，这里执行的就是“voice_control_move.py”和“init_pose.py”文件的内容。

<output>则是发送的目标，这里的发送目标是屏幕。

```
<node name="init_pose" pkg="hiwonder_slam" type="init_pose.py" output="screen"/>
<node pkg="xf_mic_asr_offline" type="voice_control_move.py" name="voice_control_move_node" output="screen"/>
```

2.5.2. Python 程序

程序源码位于：

/home/hiwonder/ros_ws/src/xf_mic_asr_offline/scripts/voice_control_move.py

```
22 class VoiceControlNode:
23     def __init__(self, name):
24         rospy.init_node(name, anonymous=True)
25
26         self.angle = None
27         self.words = None
28         self.running = True
29         self.haved_stop = False
30         self.lidar_follow = False
31         self.start_follow = False
32         self.last_status = Twist()
33         self.threshold = 3
34         self.speed = 0.3
35         self.stop_dist = 0.4
36         self.count = 0
37         self.scan_angle = math.radians(20)
38
39         self.pid_yaw = pid.PID(1.6, 0, 0.16)
40         self.pid_dist = pid.PID(1.7, 0, 0.16)
```

◆ 订阅服务

订阅获取语音识别词和唤醒角度的服务，连接底层驱动化的节点。

```
46 self.mecanum_pub = rospy.Publisher('/hiwonder_controller/cmd_vel', Twist, queue_size=1)
47 self.lidar_sub = rospy.Subscriber('/scan', sensor_msgs.LaserScan, self.lidar_callback)
48 rospy.Subscriber('/voice_words', String, self.words_callback)
49 rospy.Subscriber('/mic/awake/angle', Int32, self.angle_callback)
```

◆ 唤醒小车

根据唤醒词，匹配成功后，通过 voice_play 模块播放对应的语音。

```
65 self.words = json.dumps(msg.data, ensure_ascii=False)[1:-1]
66 print('words:', self.words)
67 if self.words is not None and self.words not in ['唤醒成功', '休眠', '失败5次', '失败10次']:
68     pass
69 elif self.words == '唤醒成功':
70     voice_play.play('awake')
```

◆ 打印声源角度

根据麦克风阵列识别到的声源角度，计算出机器人转向的角度，并打印出来。

```
78 self.angle = msg.data + 90
79 print('angle:', self.angle)
80 self.start_follow = False
```

◆ 雷达避障

根据激光雷达的扫描数据，获取并比较左右两边的距离，判断机器人转动的方向和角度。

```
88 max_index = int(math.radians(MAX_SCAN_ANGLE / 2.0) / lidar_data.angle_increment)
89 left_ranges = lidar_data.ranges[:max_index]
90 right_ranges = lidar_data.ranges[::-1][:max_index]
91 if self.start_follow:
92     angle = self.scan_angle / 2
93     angle_index = int(angle / lidar_data.angle_increment + 0.50)
94     left_ranges, right_ranges = left_ranges[:angle_index], right_ranges[:angle_index]
95     ranges = list(right_ranges[::-1])
96     ranges.extend(left_ranges)
97     min_index = np.argmin(np.array(ranges)) # 找出距离最小值
98     dist = ranges[min_index]
99     angle = -angle + lidar_data.angle_increment * min_index # 计算最小值对应的角度
100 if dist < self.threshold and abs(math.degrees(angle)) > 5: # 控制左右
101     self.pid_yaw.update(-angle)
102     twist.angular.z = misc.set_range(self.pid_yaw.output, -self.speed * 6, self.speed * 6)
103 else:
104     self.pid_yaw.clear()
105
106 if dist < self.threshold and abs(self.stop_dist - dist) > 0.02:
107     self.pid_dist.update(self.stop_dist - dist)
108     twist.linear.x = misc.set_range(self.pid_dist.output, -self.speed, self.speed)
```

◆ 语音控制

匹配识别的语句，通过 voice_play 模块播放对应的语音，再根据识别内容，设定对应的线速度或角速度，让底层驱动节点 mecanum_pub 发布消息，控制机器人移动。

```
126         if self.words == '前进':
127             voice_play.play('go')
128             self.time_stamp = rospy.get_time() + 2
129             twist.linear.x = 0.2
130         elif self.words == '后退':
131             voice_play.play('back')
132             self.time_stamp = rospy.get_time() + 2
133             twist.linear.x = -0.2
134         elif self.words == '左转':
135             voice_play.play('turn_left')
136             self.time_stamp = rospy.get_time() + 2
137             twist.angular.z = 0.8
138         elif self.words == '右转':
139             voice_play.play('turn_right')
140             self.time_stamp = rospy.get_time() + 2
141             twist.angular.z = -0.8
142         elif self.words == '漂移':
143             voice_play.play('drift')
144             twist.linear.y = 0.2
145             twist.angular.z = -0.5
146             self.time_stamp = rospy.get_time() + 2*math.pi/0.5
147         elif self.words == '过来':
148             voice_play.play('come')
149             if self.angle > 180:
150                 twist.angular.z = 1
151                 self.time_stamp = rospy.get_time() + math.radians(360 - self.angle)
152             else:
153                 twist.angular.z = -1
154                 self.time_stamp = rospy.get_time() + math.radians(self.angle)
155             self.lidar_follow = True
```

2.6. 功能延伸

2.6.1. 修改唤醒词

程序默认的唤醒词是“hello hiwonder”，可以通过修改配置文件来更改唤醒词。此处以将唤醒词替换为“小爱小爱”为例，具体步骤如下：

注意：输入指令时需要严格区分大小写，且可使用“Tab”键补齐关键词。

1) 启动机器人，将其连接至远程控制软件 NoMachine。

2) 点击系统桌面的图标，打开命令行终端。

3) 输入指令，并按下回车，进入配置文件目录。

```
roscd xf_mic_asr_offline/launch/
```

```
hiwonder@hiwonder:~$ roscd xf_mic_asr_offline/launch/
```

4) 输入指令，并按下回车，打开配置文件。

```
vim mic_init.launch
```

```
hiwonder@hiwonder:~/ros_ws/src/xf_mic_asr_offline/launch$ vim mic_init.launch
```

5) 找到下图所示代码:

```
6 <arg name="chinese_awake_words" default="小爱小爱"/>
```

6) 按下“i”键, 进入编辑模式。将“chinese_awake_words”的值修改为“小爱小爱”。

```
6 <arg name="chinese_awake_words" default="小爱小爱"/>
```

7) 修改完成后, 按下“Esc”键, 输入“:wq”, 并按下回车, 保存并退出文件。

```
:wq
```

8) 参照“[3.实验步骤](#)”重新开启玩法, 即可查看修改后的玩法效果。

3. 语音控制机械臂

3.1. 实验简介

本小节实验通过机器人搭载的语音识别功能结合视觉机械臂, 控制机械臂执行对应的动作。

在程序设计上, 通过订阅麦克风阵列节点发布的语音识别服务, 对语音进行定位、降噪、识别等操作后, 获取识别的语句和声源的角度。接着, 成功唤醒机器人, 说出特定语句后, 机器人会进行相应的语音反馈回答, 同时去识别到特定的颜色后, 通过对机器人发布的语音指令, 控制机械臂做出对应动作。

我们需要参考后文的“准备工作”, 完成本次实验的准备工作, 接着参考“操作步骤”学习体验本次实验。

3.2. 准备工作

1) 在本节课开始前, 需要将语音模块安装至机器人上(如已安装可跳过), 并连接至

集线器的 USB 接口；

2) 请参考本目录下的“[1. 六路麦克风阵列配置工作\(必读必做!\)](#)”参看 Jetson Nano 部分完成对 APPID 的申请和对文件的替换。

3) 系统出厂默认使用英文的唤醒词“hello hiwonder”进行唤醒，如果需要更换语言为中文或使用的是“AI 语音盒子 (WonderEchoPro)”需要烧录语音交互应用指令词，参考同目录下“\语音交互应用\中英文唤醒词切换”资料进行语言切换或指令词烧录。

3.3. 实验步骤

注意：输入指令时需要严格区分大小写，且可使用“Tab”键补齐关键词。

1) 启动机器人，将其连接至远程控制软件 NoMachine。关于如何连接可参考“1 教程资料\1 机器人快速使用指南\配套手册\1. JetRover 使用入门->4. 开发环境搭建”。

2) 点击系统桌面的图标，打开命令行终端。

3) 输入指令，并按下回车，关闭手机 APP 服务。

```
sudo systemctl stop start_app_node.service
```

```
> sudo systemctl stop start_app_node.service
```

4) 输入指令，并按下回车，开启语音控制小车机械臂功能：

```
roslaunch xf_mic_asr_offline voice_control_arm.launch
```

```
~ roslaunch xf_mic_asr_offline voice_control_arm.launch
```

5) 等待程序加载成功之后，先说说唤醒词“小幻小幻”，等待扬声器播放“我在”，然后接着说出控制指令“拔个萝卜”，机械臂会夹去正前方的物体；我们说“拿给我”，机械臂会将夹取到的物体从侧面递给使用者；

注意：

1. 为保证良好的体验效果，请在相对安静的环境下进行。

2. 在每次说语音指令前，建议先说唤醒词。

3. 说语音指令时声音要洪亮、清晰。

4. 语音指令要一条一条进行，在机器人执行完一次反馈后再说出其它指令。

7) 如需关闭此玩法，可在终端界面按下“Ctrl+C”。若关闭失败，请反复尝试。

8) 若后续想要重新体验有关手机 APP 的玩法，则需要输入指令重启相关服务。输入并回车执行指令后，等待机械臂恢复至初始姿态即可。

```
sudo systemctl start start_app_node.service
```

```
└─ sudo systemctl start start_app_node.service
```

3.4. 程序分析

语音控制机械臂，是让语音控制节点跟机器人底层驱动节点建立联系，在通过声音发出的命令来控制机械臂按命令执行对应动作。

3.4.1. launch 启动文件

该 launch 启动文件路径位于：

```
/home/hiwonder/ros_ws/src/xf_mic_asr_offline/launch/voice_control_arm.launch.py
```

◆ 启动多个 launch 文件

<include>标签在这里用于包含其他的 launch 文件，在该文件被启动时，会将其他的 launch 文件一同启动。

这里主要启动了麦克风的初始化以及舵机控制的启动文件。

```
<include file="$(find xf_mic_asr_offline)/launch/mic_init.launch"/>
<include file="$(find ros_robot_controller)/launch/ros_robot_controller_node.launch"/>
```

◆ 启动节点

<node>为文件中启动的节点，后续的<name>、<pkg>、<type>、<output>都为该节点内的参数。

<name>是该节点名字，<pkg>就是该节点所在的功能包；

<type>是该节点执行的文件名称，这里执行的就是：

“voice_control_color_track.py” 文件的内容；

<output>则是发送的目标，这里的发送目标是屏幕。

```
<node pkg="xf_mic_asr_offline" type="voice_control_arm.py" name="voice_control_arm_node" output="screen"/>
```

3.4.2. Python 启动文件

程序源码位于：

/home/hiwonder/ros_ws/src/xf_mic_asr_offline/scripts/voice_control_arm.py

```
def __init__(self, name):
    rospy.init_node(name, anonymous=True)

    self.language = os.environ['ASR_LANGUAGE']
    self.mic_type = os.environ['MIC_TYPE']
    rospy.Subscriber('/asr_node/voice_words', String, self.words_callback)
    self.controller = ActionGroupController(use_ros=True)
    rospy.sleep(1)
    self.controller.runAction('init')
    rospy.wait_for_service('/voice_control/get_offline_result')
    self.play('running')
    self.buzzer_pub = rospy.Publisher('/ros_robot_controller/set_buzzer', BuzzerState, queue_size=1)
    rospy.loginfo('唤醒口令: 小幻小幻(Wake up word: hello hiwonder)')
    rospy.loginfo('唤醒后15秒内可以不用再唤醒(No need to wake up within 15 seconds after waking up)')
    rospy.loginfo('控制指令: 拔个萝卜 拿给我(Voice command: pick a carrot/pass me please)')
```

◆ 环境变量获取

```
self.language = os.environ['ASR_LANGUAGE']
self.mic_type = os.environ['MIC_TYPE']
```

从系统的环境变量中获取配置项 ASR_LANGUAGE 和 MIC_TYPE，分别代表语音识别语言和麦克风类型。

◆ 创建订阅者

```
rospy.Subscriber('/asr_node/voice_words', String, self.words_callback)
```

监听 /asr_node/voice_words 话题，接收到消息时调用 self.words_callback 方法进行处理。

◆ 创建发布者

```
self.buzzer_pub = rospy.Publisher('/ros_robot_controller/set_buzzer', BuzzerState, queue_size=1)
```

发布到 /ros_robot_controller/set_buzzer 话题，消息类型是 BuzzerState。

◆ 定义回调方法

```
def words_callback(self, msg):
    if self.mic_type == 'xf':
        words = json.dumps(msg.data, ensure_ascii=False)[1:-1]
        if self.language == 'Chinese':
            words = words.replace(' ', '')
        elif self.mic_type == 'WonderEchoPro':
            words = msg.data
        print('words:', words)
    if words is not None and words not in ['唤醒成功(wake-up-success)', '休眠(sleep)', '失败5次(Fail-5-times)', '失败10次(Fail-10-times)']:
        if words == '拔个萝卜' or words == 'pick a carrot':
            self.play('ok')
            self.controller.runAction('voice_pick')
        elif words == '拿给我' or words == 'pass me please':
            self.play('come')
            self.controller.runAction('voice_give')
        elif words == '唤醒成功(wake-up-success)':
            self.play('awake')
        elif words == '休眠(sleep)':
            msg = BuzzerState()
            msg.freq = 1900
            msg.on_time = 0.05
            msg.off_time = 0.01
            msg.repeat = 1
            self.buzzer_pub.publish(msg)
```

识别到关键词后，执行对应动作组。

4. 语音控制颜色识别

本节课是通过语音命令来开启或关闭颜色识别功能，当识别到红绿蓝任意一种颜色，扬声器则播报对应颜色。

4.1. 实验简介

首先，订阅麦克风阵列节点发布的语音识别服务，对语音进行定位、降噪、识别等操作后，获取识别的语句和声源的角度。

随后，用特定唤醒词唤醒机器人。说出唤醒词后，机器人会进行相应的语音反馈回答。

最后再发布相应的命令，机器人会根据命令开启或关闭颜色识别。

颜色识别的实现分为颜色识别和识别后的执行反馈两部分。

首先颜色识别部分，先进行高斯滤波（对图像中进行减噪），再通过 Lab 颜色空间来对物品的颜色进行转换。

接着通过颜色阈值对圆圈中的物体颜色进行识别，再对图像部分进行掩膜（掩膜是用选定的图像、图形或物体，对处理的图像进行全局或者局部遮挡）。

然后经过对物品图片开闭运算处理后，最后将轮廓最大的物体用圆圈圈出。

4.2. 准备工作

- 1) 在本节课开始前，需要将语音模块安装至机器人上（如已安装可跳过），并连接至集线器的 USB 接口；
- 2) 请参考本目录下的“[1.六路麦克风阵列配置工作（必读必做！）](#)”参看 Jetson Nano 部分完成对 APPID 的申请和对文件的替换。
- 3) 系统出厂默认使用英文的唤醒词“hello hiwonder”进行唤醒，如果需要更换语言为中文或使用的是“AI 语音盒子（WonderEchoPro）”需要烧录语音交互应用指令词，参考同目录下“\语音交互应用\中英文唤醒词切换”资料进行语言切换或指令词烧录。

4.3. 实验步骤

注意：

- 1、输入指令时需要严格区分大小写，且可使用“Tab”键补齐关键词。
- 2、识别色块时要注意背景里不要出现与色块颜色相似或相同的物体，避免产生干扰。
- 3、如果颜色识别不准确，可以前往“1 教程资料\3 ROS1 课程\7 ROS+OpenCV 课程\配套手册\ROS+OpenCV 课程”调节颜色阈值。

- 1) 启动机器人，将其连接至远程控制软件 NoMachine。关于如何连接可参考“1 教程资料\1 机器人快速使用指南\配套手册\1.JetRover 使用入门->4.开发环境搭建”。

- 2) 点击系统桌面的图标，打开命令行终端。

- 3) 输入指令，并按下回车，关闭手机 APP 服务。

```
sudo systemctl stop start_app_node.service
```

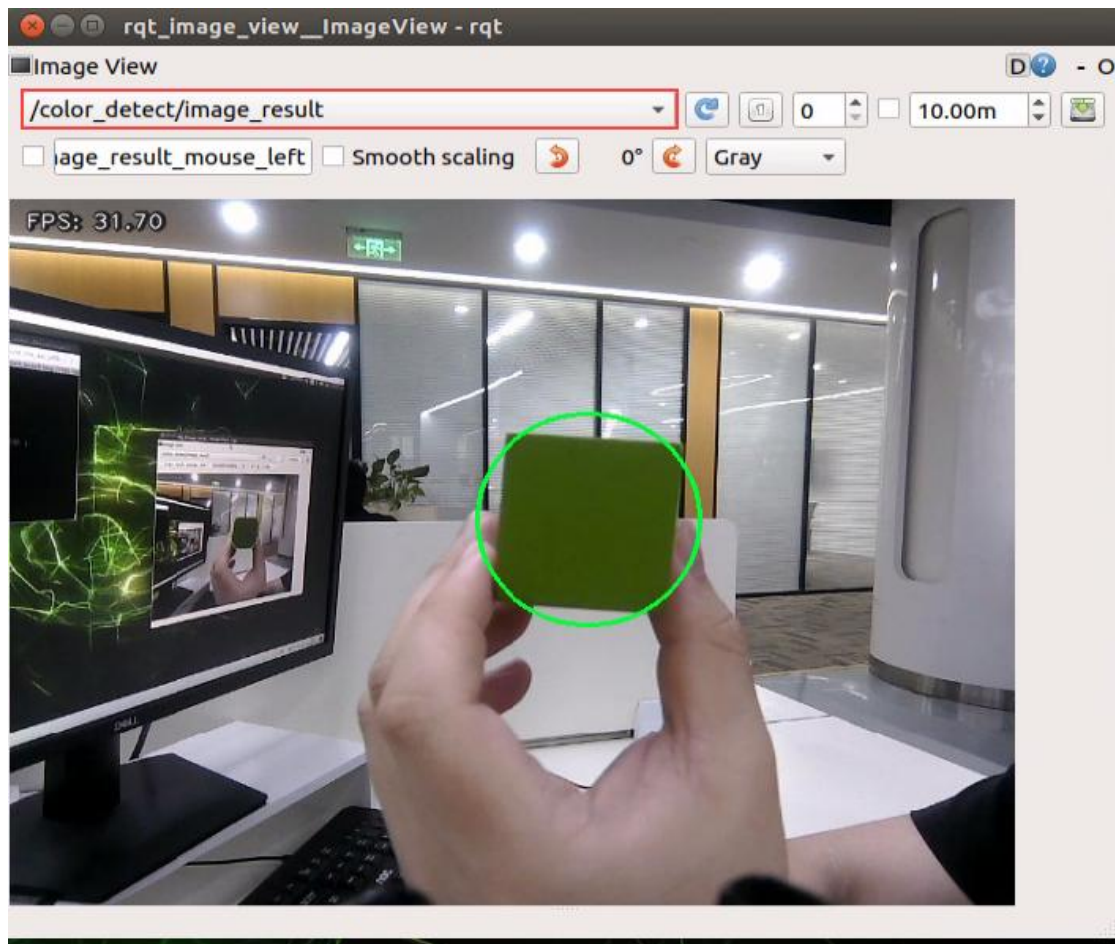
```
hiwonder@hiwonder:~$ sudo systemctl stop start_app_node.service
```

- 4) 输入指令，并按下回车，开启玩法。

```
roslaunch xf_mic_asr_offline voice_control_color_detect.launch
```

```
hiwonder@hiwonder:~$ roslaunch xf_mic_asr_offline voice_control_color_detect.launch
```

5) 若要打开回传画面，则需打开一个新的终端，在终端内输入“`rqt_image_view`”，打开可视化工具，并选择颜色识别分拣的信道“`/color_detect/image_result`”即可。



6) 若想关闭玩法，则在各个终端内按下“`Ctrl+C`”关闭玩法即可。若关闭失败，请反复尝试。

7) 若后续想要重新体验有关手机 APP 的玩法，则需要输入指令重启相关服务。输入并回车执行指令后，等待机械臂恢复至初始姿态即可。

```
sudo systemctl start start_app_node.service
```

```
hiwonder@hiwonder:~$ sudo systemctl start start_app_node.service
```

4.4. 实现效果

注意：

1. 为保证良好的体验效果，请在相对安静的环境下进行。

2. 在每次说语音指令前，建议先说唤醒词。

3. 说语音指令时声音要洪亮、清晰。

4. 语音指令要一条一条进行，在机器人执行完一次反馈后再说出其它指令。

启动玩法后，先说唤醒词“hello hiwonder”，再说指令“开启颜色识别”，即可令机器人开始识别颜色。

此玩法的可识别颜色有红色、绿色和蓝色。当识别到颜色，机器人会播报颜色名称。以红色为例，将红色方块放置在摄像头的视野范围内，当识别到色块，机器人播报语音“红色”。

如需令机器人停止识别颜色，先说唤醒词“hello hiwonder”，再说指令“关闭颜色识别”即可。

4.5. 程序分析

语音控制颜色识别，是让语音控制节点跟机器人底层驱动节点、摄像头节点建立联系，在通过声音发出的命令来控制机器人对色块进行识别。

4.5.1. launch 启动文件

该 launch 启动文件路径位于：

`/home/hiwonder/ros_ws/src/xf_mic_asr_offline/launch/voice_control_color_detect.launch`

◆ 启动多个 launch 文件

<include>标签在这里用于包含其他的 launch 文件，在该文件被启动时，会将其他的 launch 文件一同启动。

这里主要是启动了麦克风的初始化、底盘控制的驱动以及颜色识别的启动文件。

```
<launch>
  <include file="$(find xf_mic_asr_offline)/launch/mic_init.launch"/>
  <!-- 底盘驱动 (chassis driver) -->
  <include file="$(find hiwonder_controller)/launch/hiwonder_controller.launch"/>
  <node name="init_pose" pkg="hiwonder_slam" type="init_pose.py" output="screen">
    <param name="horizontal" value="true"/>
  </node>
  <include file="$(find hiwonder_example)/scripts/color_detect/color_detect_node.launch"/>
  <node pkg="xf_mic_asr_offline" type="voice_control_color_detect.py" name="voice_control_color_detect_node" output="screen"/>
</launch>
```

◆ 启动节点

<node>为文件中启动的节点，后续的<name>、<pkg>、<type>、<output>等都为该节点内的参数。

<name>是该节点名字，<pkg>就是该节点所在的功能包；

<type>是该节点执行的文件名称，这里执行的就是：

“voice_control_color_detect.py”和“init_pose.py”文件的内容。

<output>则是发送的目标，这里的发送目标是屏幕。

<param>在 launch 文件中是参数标签，储存在参数服务器中。其中的内容就是声明的参数，后面的<default>则是对该参数进行了具体定义，

```
<launch>
  <include file="$(find xf_mic_asr_offline)/launch/mic_init.launch"/>
  <!--底盘驱动(chassis driver)-->
  <include file="$(find hiwonder_controller)/launch/hiwonder_controller.launch"/>
  <node name="init_pose" pkg="hiwonder_slam" type="init_pose.py" output="screen">
    <param name="horizontal" value="True"/>
  </node>
  <include file="$(find hiwonder_example)/scripts/color_detect/color_detect_node.launch"/>
  <node pkg="xf_mic_asr_offline" type="voice_control_color_detect.py" name="voice_control_color_detect_node" output="screen"/>
</launch>
```

4.5.2. Python 程序

程序源码位于：

/home/hiwonder/ros_ws/src/xf_mic_asr_offline/scripts/voice_control_color_detect.py

```
14 class VoiceControlColorDetectNode:
15     def __init__(self, name):
16         rospy.init_node(name, anonymous=True)
17         self.count = 0
18         self.color = None
19         self.running = True
20         self.last_color = None
21         signal.signal(signal.SIGINT, self.shutdown)
22
23         rospy.Subscriber('/voice_words', String, self.words_callback)
24         rospy.Subscriber('/color_detect/color_info', ColorInfo, self.get_color_callback)
25         voice_play.play('running')
26
27         rospy.loginfo('唤醒口令: 小幻小幻')
28         rospy.loginfo('唤醒后15秒内可以不用再唤醒')
29         rospy.loginfo('控制指令: 开启颜色识别 关闭颜色识别')
30
31     while self.running:
32         if self.color == 'red' and self.last_color != 'red':
33             self.last_color = 'red'
34             voice_play.play('red')
35         elif self.color == 'green' and self.last_color != 'green':
36             self.last_color = 'green'
37             voice_play.play('green')
38         elif self.color == 'blue' and self.last_color != 'blue':
39             self.last_color = 'blue'
40             voice_play.play('blue')
41         else:
42             self.count += 1
43             rospy.sleep(0.01)
44             if self.count > 50:
45                 self.count = 0
46                 self.last_color = self.color
47             rospy.signal_shutdown('shutdown')
```

◆ 订阅服务

订阅获取颜色识别节点。

```
23 rospy.Subscriber('/voice_words', String, self.words_callback)
24 rospy.Subscriber('/color_detect/color_info', ColorInfo, self.get_color_callback)
25 voice_play.play('running')
26
27 rospy.loginfo('唤醒口令: 小幻小幻')
28 rospy.loginfo('唤醒后15秒内可以不用再唤醒')
29 rospy.loginfo('控制指令: 开启颜色识别 关闭颜色识别')
```

◆ 设置唤醒词和控制语音

设置唤醒词和控制语音。

```
23     rospy.Subscriber('/voice_words', String, self.words_callback)
24     rospy.Subscriber('/color_detect/color_info', ColorInfo, self.get_color_callback)
25     voice_play.play('running')
26
27     rospy.loginfo('唤醒口令: 小幻小幻')
28     rospy.loginfo('唤醒后15秒内可以不用再唤醒')
29     rospy.loginfo('控制指令: 开启颜色识别 关闭颜色识别')
```

◆ 唤醒小车

根据唤醒词，匹配成功后，通过 `open_success` 模块播放对应的语音。

```
59     def words_callback(self, msg):
60         words = json.dumps(msg.data, ensure_ascii=False)[1:-1]
61         print('words:', words)
62         if words is not None and words not in ['唤醒成功', '休眠', '失败5次', '失败10次']:
63             if words == '开启颜色识别':
64                 res = rospy.ServiceProxy('/color_detect/set_color', SetStringList)(['red', 'green', 'blue'])
65                 if res.success:
66                     voice_play.play('open_success')
```

◆ 语音控制

匹配识别的语句，通过 `voice_play` 模块播放对应的语音，再根据识别内容，设定对应的颜色，播报相应的语音。

```
31     while self.running:
32         if self.color == 'red' and self.last_color != 'red':
33             self.last_color = 'red'
34             voice_play.play('red')
35         elif self.color == 'green' and self.last_color != 'green':
36             self.last_color = 'green'
37             voice_play.play('green')
38         elif self.color == 'blue' and self.last_color != 'blue':
39             self.last_color = 'blue'
40             voice_play.play('blue')
41         else:
42             self.count += 1
43             rospy.sleep(0.01)
44             if self.count > 50:
45                 self.count = 0
46                 self.last_color = self.color
47             rospy.signal_shutdown('shutdown')
```

5. 语音控制颜色追踪

本节课是通过发布语音命令来控制机器人对特定的颜色进行追踪,同时对语音控制和颜色追踪的原理进行介绍。

5.1. 实验简介

首先, 订阅麦克风阵列节点发布的语音识别服务, 对语音进行定位、降噪、识别等操作后, 获取识别的语句和声源的角度。

随后, 用特定唤醒词唤醒机器人。说出唤醒词后, 机器人会进行相应的语音反馈回答。

最后再发布相应的命令, 机器人会根据命令对指定的颜色进行追踪。

目标追踪的实现可分为颜色识别和追踪两部分。

首先颜色识别部分, 先进行高斯滤波(对图像中进行减噪), 通过 Lab 颜色空间来对物品的颜色进行转换(关于 Lab 颜色空间的这个部分可以前往“4 OpenCV 基础课程”进行详细学习)。

接着通过颜色阈值对圆圈中的物体颜色进行识别, 再对图像部分进行掩膜(掩膜是用选定的图像、图形或物体, 对处理的图像进行全局或者局部遮挡)。

然后经过对物品图片开闭运算处理后, 最后将轮廓最大的物体用圆圈圈出。

追踪部分使用 PID 算法, 根据目标的画面像素坐标与画面中心坐标进行比较, 减小两个坐标之间的距离, 从而实现目标追踪。

PID 算法是应用最为广泛的一种自动控制器。在过程控制中, 按误差的比例(P)、积分(I)和微分(D)进行控制。它具有原理简单, 易于实现, 适用面广, 控制参数相互独立, 参数的选定比较简单等优点。

5.2. 准备工作

1) 在本节课开始前, 需要将语音模块安装至机器人上(如已安装可跳过), 并连接至集线器的 USB 接口;

2) 请参考本目录下的“[1.六路麦克风阵列配置工作\(必读必做!\)](#)”参看 Jetson Nano 部分完成对 APPID 的申请和对文件的替换。

3) 系统出厂默认使用英文的唤醒词“hello hiwonder”进行唤醒，如果需要更换语言为中文或使用的是“AI 语音盒子 (WonderEchoPro)”需要烧录语音交互应用指令词，参考同目录下“\语音交互应用\中英文唤醒词切换”资料进行语言切换或指令词烧录。

5.3. 实验步骤

注意：

- 1、输入指令时需要严格区分大小写，且可使用“Tab”键补齐关键词。
- 2、识别色块时要注意背景里不要出现与色块颜色相似或相同的物体，避免产生干扰。
- 3、如果颜色识别不准确，可以前往“第十一章 ROS+OpenCV 视觉识别标准课程/第 1 课 颜色阈值调节”调节颜色阈值。

1) 启动机器人，将其连接至远程控制软件 NoMachine。关于如何连接可参考“1 JetRover 使用入门”。

2) 点击系统桌面的图标，打开命令行终端。

3) 执行指令，关闭手机 APP 服务。

```
sudo systemctl stop start_app_node.service
```

```
hiwonder@hiwonder:~$ sudo systemctl stop start_app_node.service
```

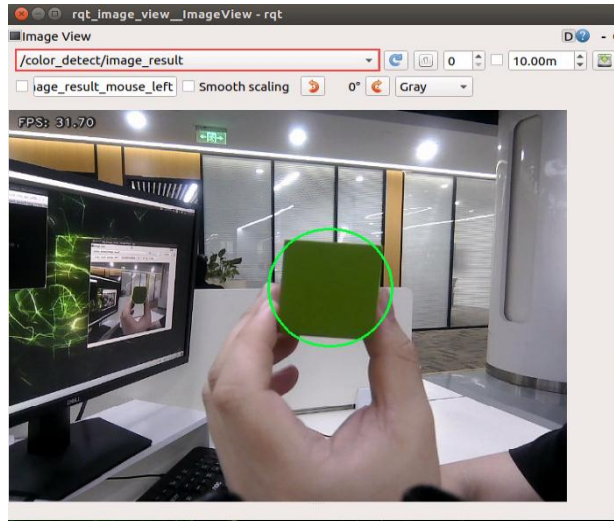
4) 输入指令，并按下回车，开启玩法。

```
roslaunch xf_mic_asr_offline voice_control_color_track.launch
```

```
hiwonder@hiwonder:~$ roslaunch xf_mic_asr_offline voice_control_color_track.launch
```

5) 若想关闭玩法，则在终端内按下“Ctrl+C”关闭玩法即可。若关闭失败，请反复尝试。

6) 若要打开回传画面，则需打开一个新的终端，在终端内输入“rqt_image_view”，打开可视化工具，并选择颜色识别分拣的信道“/color_detect/image_result”即可。



7) 若后续想要重新体验有关手机 APP 的玩法，则需要输入指令重启相关服务。输入并回车执行指令后，等待机械臂恢复至初始姿态即可。

```
sudo systemctl start start_app_node.service
```

```
hiwonder@hiwonder:~$ sudo systemctl start start_app_node.service
```

5.4. 实现效果

注意：

1. 为保证良好的体验效果，请在相对安静的环境下进行。
2. 在每次说语音指令前，建议先说唤醒词。
3. 说语音指令时声音要洪亮、清晰。
4. 语音指令要一条一条进行，在机器人执行完一次反馈后再说出其它指令。

启动玩法后，先说唤醒词“hello hiwonder”，再说追踪指令，即可令机器人开始追踪颜色。

此玩法的可识别颜色有红色、绿色和蓝色。以红色为例，将红色方块放置在摄像头的视野范围内。先说唤醒词“hello hiwonder”，再说追踪指令“追踪红色”。随后，当识别到色块，机器人会转动舵机，使深度相机正对红色方块。移动色块，云台也会随之进行转动。

如需令机器人停止追踪颜色，先说唤醒词“hello hiwonder”，再说指令“停止追踪”

即可。

5.5. 程序分析

语音控制颜色追踪，是让语音控制节点跟摄像头节点建立联系，在通过声音发出的命令来控制机器人对色块进行识别并追踪。

5.5.1. launch 启动文件

该 launch 启动文件路径位于：

`/home/hiwonder/ros_ws/src/xf_mic_asr_offline/launch/voice_control_color_track.launch`

◆ 启动多个 launch 文件

`<include>`标签在这里用于包含其他的 launch 文件，在该文件被启动时，会将其他的 launch 文件一同启动。

这里主要启动了麦克风的初始化以及颜色追踪的启动文件。

```
<include file="$(find xf_mic_asr_offline)/launch/mic_init.launch"/>
<include file="$(find hiwonder_example)/scripts/color_track/color_track_node.launch">
  <arg name="start" value="false"/>
</include>
```

◆ 启动节点

`<node>`为文件中启动的节点，后续的`<name>`、`<pkg>`、`<type>`、`<output>`都为该节点内的参数。

`<name>`是该节点名字，`<pkg>`就是该节点所在的功能包；

`<type>`是该节点执行的文件名称，这里执行的就是：

“`voice_control_color_track.py`”文件的内容；

`<output>`则是发送的目标，这里的发送目标是屏幕。

```
<node pkg="xf_mic_asr_offline" type="voice_control_color_track.py" name="voice_control_color_track_node" output="screen"/>
```

5.5.2. Python 程序

◆ 订阅服务

Python 程序的位于：

`/home/hiwonder/ros_ws/src/xf_mic_asr_offline/scripts/voice_control_color_track.py`

◆ 订阅获取颜色识别节点和舵机控制节点以及设置唤醒词和控制语音。

```
35 self.joints_pub = rospy.Publisher('/servo_controllers/port_id_1/multi_id_pos_dur', MultiRawIdPosDur, queue_size=1) # 舵机控制
36 self.mecanum_pub = rospy.Publisher('/hiwonder_controller/cmd_vel', Twist, queue_size=1) # 底盘控制
37
38 rospy.Subscriber('/color_detect/color_info', ColorsInfo, self.get_color_callback)
39
40 rospy.Service('~start', Trigger, self.start_srv_callback) # 进入玩法
41 rospy.Service('~stop', Trigger, self.stop_srv_callback) # 退出玩法
42 rospy.Service('~set_color', SetString, self.set_color_srv_callback) # 设置颜色
```

◆ 唤醒机器人

根据唤醒词，匹配成功后，通过 voice_play 模块播放对应功能的语音。

```
49 words = json.dumps(msg.data, ensure_ascii=False)[1:-1]
50 print('words:', words)
51 if words is not None and words not in ['唤醒成功', '休眠', '失败5次', '失败10次']:
52     if words == '追踪红色':
53         res = rospy.ServiceProxy('/color_detect/set_color', SetStringList)(['red'])
54         if res.success:
55             voice_play.play('start_track_red')
56         else:
57             voice_play.play('track_fail')
58     elif words == '追踪绿色':
59         res = rospy.ServiceProxy('/color_detect/set_color', SetStringList)(['green'])
60         if res.success:
61             voice_play.play('start_track_green')
62         else:
63             voice_play.play('track_fail')
64     elif words == '追踪蓝色':
65         res = rospy.ServiceProxy('/color_detect/set_color', SetStringList)(['blue'])
66         if res.success:
67             voice_play.play('start_track_blue')
68         else:
69             voice_play.play('track_fail')
76 elif words == '唤醒成功':
77     voice_play.play('awake')
```

◆ 语音控制

通过颜色识别节点，将深度相机回传画面内的颜色识别出来，然后通过物体在回传画面的中的位置，使用 PID 调整舵机的角度，使深度相机正对物体。

```
83 def color_track(self):
84     while self.running:
85         if self.center is not None:
86             self.pid.SetPoint = self.center.width/2
87             self.pid.update(self.center.x)
88             self.x_dis += self.pid.output
89             if self.x_dis > 1000:
90                 self.x_dis = 1000
91             if self.x_dis < 0 :
92                 self.x_dis = 0
93             set_servos(self.joints_pub, 20, ((5, int(self.x_dis)), ))
94             rospy.sleep(0.02)
95         else:
96             rospy.sleep(0.01)
97
98     set_servos(self.joints_pub, 1000, ((5, 500), ))
99     rospy.signal_shutdown('shutdown')
```

6. 语音控制色块分拣

本节课是通过语音控制机器人开启色块分拣玩法, 识别到色块后将其放置到对应的位置上, 同时还将介绍其原理。

6.1. 实验简介

1) 首先, 订阅麦克风阵列节点发布的语音识别服务, 对语音进行定位、降噪、识别等操作后, 获取识别的语句和声源的角度。

接着, 成功唤醒机器人, 说出特定语句后, 机器人会进行相应的语音反馈回答。

最后再发布相应的命令, 机器人会根据命令开启或关闭对色块的分拣。

2) 色块分拣分为 ROI 区域划分、颜色识别和分拣三部分。

首先是 ROI 区域划分, 在摄像头的视野范围内选定一个 ROI 区域, 在这个区域里面会对指定的颜色进行颜色识别, 也就是我们在开启玩法后看到的回传画面的黄色框。

ROI 即为感兴趣区域, 在图像处理方面, 从被处理的图像以方框、圆、椭圆等方式来勾勒需要处理的区域。

颜色识别部分, 先进行高斯滤波 (对图像中进行减噪), 通过 Lab 颜色空间来对物品

的颜色进行转换。

接着通过颜色阈值对圆圈中的物体颜色进行识别，再对图像部分进行掩膜（掩膜是用选定的图像、图形或物体，对处理的图像进行全局或者局部遮挡）。

然后经过对物品图片开闭运算处理后，最后将轮廓最大的物体用圆圈圈出。

3) 识别到特定的颜色后,机械臂将会下降到指定位置将色块夹起,接着放到指定位置。

6.2. 准备工作

1) 在本节课开始前，需要将语音模块安装至机器人上（如已安装可跳过），并连接至集线器的 USB 接口；

2) 请参考本目录下的“[1.六路麦克风阵列配置工作\(必读必做!\)](#)”参看 Jetson Nano 部分完成对 APPID 的申请和对文件的替换。

3) 系统出厂默认使用英文的唤醒词“hello hiwonder”进行唤醒，如果需要更换语言为中文或使用的是“AI 语音盒子 (WonderEchoPro)”需要烧录语音交互应用指令词，参考同目录下“\语音交互应用\中英文唤醒词切换”资料进行语言切换或指令词烧录。

6.3. 实验步骤

注意：

- 1、输入指令时需要严格区分大小写，且可使用“Tab”键补齐关键词。
 - 2、识别色块时要注意背景里不要出现与色块颜色相似或相同的物体，避免产生干扰。
 - 3、如果颜色识别不准确，可以前往“第十一章 ROS+OpenCV 视觉识别标准课程/第 1 课 颜色阈值调节”调节颜色阈值。
-

1) 启动机器人，将其连接至远程控制软件 NoMachine。关于如何连接可参考“1 教程资料\1 机器人快速使用指南\配套手册\1.JetRover 使用入门->4.开发环境搭建”。

2) 点击系统桌面的图标，打开命令行终端。

- 3) 输入指令，并按下回车，关闭手机 APP 服务。

```
sudo systemctl stop start_app_node.service
```

```
hiwonder@hiwonder:~$ sudo systemctl stop start_app_node.service
```

- 4) 输入指令并执行开启玩法。

```
roslaunch xf_mic_asr_offline voice_control_color_sorting.launch
```

```
hiwonder@hiwonder:~$ roslaunch xf_mic_asr_offline voice_control_color_sorting.launch
```

- 5) 说出唤醒词后，再发布指令即可开始色块分拣。

- 6) 如需关闭此玩法，可在终端界面按下“Ctrl+C”。若关闭失败，请反复尝试。

- 7) 若后续想要重新体验有关手机 APP 的玩法，则需要输入指令重启相关服务。输入并回车执行指令后，等待机械臂恢复至初始姿态即可。

```
sudo systemctl start start_app_node.service
```

```
hiwonder@hiwonder:~$ sudo systemctl start start_app_node.service
```

6.4. 功能实现

启动玩法后，先说唤醒词，再说指令语句，即可控制开启色块分拣。

例如我们说唤醒词“hello hiwonder”，再说指令“开启颜色分拣”，终端就会打开回传画面并且开始在特定区域内识别色块。识别到对应色块后就会将其抓取并放置到对应位置。

6.5. 程序分析

语音控制颜色追踪，是让语音控制节点跟摄像头节点建立联系，在通过声音发出的命令来控制机器人开机或者关闭该玩法的执行。

6.5.1. launch 启动文件

该 launch 启动文件路径位于：

```
/home/hiwonder/ros_ws/src/xf_mic_asr_offline/launch/voice_control_co
```

lor_track.launch

◆ 启动多个 launch 文件

<include>标签在这里用于包含其他的 launch 文件，在该文件被启动时，会将其他的 launch 文件一同启动。

这里主要启动了麦克风的初始化以及色块分拣的启动文件。

```
<include file="$(find xf_mic_asr_offline)/launch/mic_init.launch"/>
<include file="$(find hiwonder_example)/scripts/color_sorting/color_sorting_node.launch">
  <arg name="start" value="false"/>
</include>
```

◆ 启动节点

<node>为文件中启动的节点，后续的<name>、<pkg>、<type>、<output>都为该节点内的参数。

<name>是该节点名字，<pkg>就是该节点所在的功能包；

<type>是该节点执行的文件名称，这里执行的就是：

“voice_control_color_sorting.py” 文件的内容。

<output>则是发送的目标，这里的发送目标是屏幕。

```
<node pkg="xf_mic_asr_offline" type="voice_control_color_sorting.py" name="voice_control_color_sorting_node" output="screen"/>
```

6.5.2. Python 程序

◆ 订阅服务

程序位置：

/home/hiwonder/ros_ws/src/xf_mic_asr_offline/scripts/voice_control_color_sorting.py

订阅语音识别节点 “/voice_words” 的信息，获取识别到的语音信息。

```
22 | rospy.Subscriber('/voice_words', String, self.words_callback)
```

◆ 唤醒词唤醒机器人

```
33 | rospy.loginfo('唤醒口令: 小幻小幻 (wake up word: hello hiwonder)')
```

◆ 匹配语音命令，让机器人开启或者关闭色块分拣

```
52 if words == '开启颜色分拣' or words == 'start color sorting':  
53     res = rospy.ServiceProxy('/color_sorting/start', Trigger)()  
54     if res.success:  
55         self.play('open_success')  
56     else:  
57         self.play('open_fail')  
58 elif words == '关闭颜色分拣' or words == 'stop color sorting':  
59     res = rospy.ServiceProxy('/color_sorting/stop', Trigger)()  
60     if res.success:  
61         self.play('close_success')  
62     else:  
63         self.play('close_fail')
```

7. 语音控制垃圾分拣

本节课介绍如何用语音控制机器人对垃圾卡片进行识别并分拣。

7.1. 实验简介

首先，订阅麦克风阵列节点发布的语音识别服务，对语音进行定位、降噪、识别等操作后，获取识别的语句和声源的角度。

接着，成功唤醒机器人，说出特定语句后，机器人会进行相应的语音反馈回答。

最后，进行语句匹配，根据匹配结果，机器人会执行对应的动作。

7.2. 准备工作

1) 在本节课开始前，需要将语音模块安装至机器人上（如已安装可跳过），并连接至集线器的 USB 接口；

2) 请参考本目录下的“[1.六路麦克风阵列配置工作（必读必做！）](#)”参看 Jetson Nano 部分完成对 APPID 的申请和对文件的替换。

3) 系统出厂默认使用英文的唤醒词“hello hiwonder”进行唤醒，如果需要更换语言为中文或使用的是“AI 语音盒子（WonderEchoPro）”需要烧录语音交互应用指令词，参考同目录下“\语音交互应用\中英文唤醒词切换”资料进行语言切换或指令词烧录。

7.3. 实验步骤

注意：输入指令时需要严格区分大小写，且可使用“Tab”键补齐关键词。

1) 启动机器人，将其连接至远程控制软件 NoMachine。关于如何连接可参考“1 教程资料\1 机器人快速使用指南\配套手册\1.JetRover 使用入门->4.开发环境搭建”。



2) 点击系统桌面的图标，打开命令行终端。

3) 输入指令，并按下回车，关闭手机 APP 服务。

```
sudo systemctl stop start_app_node.service
```

```
hiwonder@hiwonder:~$ sudo systemctl stop start_app_node.service
```

4) 执行开启玩法的指令

```
roslaunch xf_mic_asr_offline voice_control_garbage_classification.launch
```

```
hiwonder@hiwonder:~$ roslaunch xf_mic_asr_offline voice_control_garbage_classification.launch
```

5) 如需关闭此玩法，可在终端界面按下“Ctrl+C”。若关闭失败，请反复尝试。

6) 若后续想要重新体验有关手机 APP 的玩法，则需要输入指令重启相关服务。输入并回车执行指令后，等待机械臂恢复至初始姿态即可。

```
sudo systemctl start start_app_node.service
```

```
hiwonder@hiwonder:~$ sudo systemctl start start_app_node.service
```

7.4. 功能实现

启动玩法后，先说唤醒词“hello hiwonder”，再说指令语句，即可控制机器人开启或关闭垃圾分类。

此玩法可以识别四种分类的垃圾，分别是厨余垃圾、有害垃圾、其它垃圾和可回收垃圾。以厨余垃圾为例。先说唤醒词“hello hiwonder”，再说追踪指令“开启垃圾分类”。随后，当识别垃圾卡片时，机械臂会下降并且夹取该卡片，并且将垃圾卡片放置到对应的垃圾区域内。

不同垃圾卡片及其对应的放置区域如下表（以机器人为第一视角）：

分类	放置区域
厨余垃圾	放置在机器人的左前方
有害垃圾	放置在机器人的左侧
其它垃圾	放置在机器人的右侧
可回收垃圾	放置在机器人的右前方

7.5. 程序分析

7.5.1. launch 启动文件

语音控制垃圾分拣，是让语音控制节点跟摄像头节点建立联系，在通过声音发出的命令来控制机器人开启或关闭该玩法，并在夹取对应的垃圾卡片后播报其分类。

该 launch 启动文件路径位于：

```
/home/hiwonder/ros_ws/src/xf_mic_asr_offline/launch/voice_control_garbage_classification.launch
```

◆ 启动多个 launch 文件

`<include>`标签在这里用于包含其他的 launch 文件，在该文件被启动时，会将其他的 launch 文件一同启动。

这里主要启动了麦克风的初始化以及垃圾分拣的启动文件。

```
<include file="$(find xf_mic_asr_offline)/launch/mic_init.launch"/>

<include file="$(find hiwonder_example)/scripts/garbage_classification/garbage_classification.launch">
  <arg name="start" value="false"/>
  <arg name="broadcast" value="true"/>
</include>
```

◆ 启动节点

<node>为文件中启动的节点，后续的<name>、<pkg>、<type>、<output>都为该节点内的参数。

<name>是该节点名字，<pkg>就是该节点所在的功能包；

<type> 是该节点执行的文件名称，这里执行的就是“voice_control_color_sorting.py”文件的内容。

<output>则是发送的目标，这里的发送目标是屏幕。

```
<node pkg="xf_mic_asr_offline" type="voice_control_garbage_classification.py" name="voice_control_garbage_classification_node" output="screen"/>
```

7.5.2. Python 程序

◆ 订阅服务

订阅获取语音识别词和唤醒角度的服务，获取识别到的语音信息。

```
21 | roslib.Subscriber('/voice_words', String, self.words_callback)
```

◆ 唤醒小车

根据唤醒词，匹配成功后，通过 voice_play 模块播放对应的语音。

```
def play(self, name):  
    voice_play.play(name, language=self.language)
```

◆ 根据命令执行对应动作

```
49 | if words == '开启垃圾分类' or words == 'sort waste':  
50 |     res = rospy.ServiceProxy('/garbage_classification/start', Trigger)()  
51 |     if res.success:  
52 |         self.play('open_success')  
53 |     else:  
54 |         self.play('open_fail')  
55 | elif words == '关闭垃圾分类' or words == 'stop sort waste':  
56 |     res = rospy.ServiceProxy('/garbage_classification/stop', Trigger)()  
57 |     if res.success:  
58 |         self.play('close_success')  
59 |     else:  
60 |         self.play('close_fail')
```

8. 语音控制多点导航

本节课将语音控制小车，然后在建好的地图上进行导航。

8.1. 实验简介

首先，打开机器人导航服务并加载地图，启动多点导航服务。

然后，订阅麦克风阵列节点发布的语音识别服务，对语音进行声源定位、降噪、识别等操作后，获取识别的语句和声源的角度。

随后，通过麦克风识别语言。识别到唤醒词和控制指令后并且满足设置的识别阈值，机器人会进行相应的语音反馈回答。

最后，根据识别对应的语言，机器人会导航到地图对应的位置，导航会先进行全局规划，行驶中遇到障碍物会进行局部规划。

8.2. 准备工作

1) 在本节课开始前，需要将语音模块安装至机器人上（如已安装可跳过），并连接至集线器的 USB 接口；

2) 请参考本目录下的“[1.六路麦克风阵列配置工作（必读必做！）](#)”参看 Jetson Nano 部分完成对 APPID 的申请和对文件的替换。

3) 系统出厂默认使用英文的唤醒词“**hello hiwonder**”进行唤醒，如果需要更换语言为中文或使用的是“**AI 语音盒子 (WonderEchoPro)**”需要烧录语音交互应用指令词，参考同目录下“**\语音交互应用\中英文唤醒词切换**”资料进行语言切换或指令词烧录。

4) 构建机器人当前所处区域的地图。如需了解如何建图，可查看目录“**教程资料\ROS1 课程\建图与导航课程**”下的文档；

5) 将机器人放置在空旷平台，确保以机器人为中心的周边 3 米内有充足的活动范围。

8.3. 实验步骤

注意：输入指令时需要严格区分大小写，且可使用“Tab”键补齐关键词。

1) 启动机器人，将其连接至远程控制软件 NoMachine。关于如何连接可参考“**1 教程资料\1 机器人快速使用指南\配套手册\1.JetRover 使用入门->4.开发环境搭建**”。



2) 点击系统桌面的图标，打开命令行终端。

3) 输入指令，并按下回车，关闭手机 APP 服务。

```
sudo systemctl stop start_app_node.service
```

```
hiwonder@hiwonder:~$ sudo systemctl stop start_app_node.service
```

4) 执行开启玩法的指令

```
roslaunch xf_mic_asr_offline voice_control_navigation.launch map:=map_01
```

```
hiwonder@hiwonder:~$ roslaunch xf_mic_asr_offline voice_control_navigation.launch map:=map_01
```

指令末端的“map_01”是地图名称，用户可根据需求修改此参数。地图的存放路径是“/home/hiwonder/ros_ws/src/hiwonder_slam/maps”。

5) 如需关闭此玩法，可在终端界面按下“Ctrl+C”。若关闭失败，请反复尝试。

6) 若后续想要重新体验有关手机 APP 的玩法，则需要输入重启相关服务。输入并回车执行指令后，等待机械臂恢复至初始姿态即可。

```
sudo systemctl start start_app_node.service
```

```
hiwonder@hiwonder:~$ sudo systemctl start start_app_node.service
```

8.4. 功能实现

启动玩法后，先说唤醒词“hello hiwonder”，再说指令语句，即可控制机器人移动。

例如先说“hello hiwonder”，机器人会回复一声“我在”。接着我们可以发布说“去 A 点”，那么机器人就会对应地去到起始位置的右上方

指令语句及其对应功能如下表（以机器人为第一视角）：

指令语句	功能
去 A 点	控制机器人移动至去 A 点（起始位置的右上方）

去 B 点	控制机器人移动至 B 点（起始位置的左上方）
去 C 点	控制机器人移动至 C 点（位于 A 点下方）
回原点	控制机器人移动至原点

8.5. 程序分析

8.5.1. launch 启动文件

该 launch 启动文件路径位于：

`/home/hiwonder/ros_ws/src/xf_mic_asr_offline/launch/voice_control_navigation.launch`

◆ 启动多个 launch 文件

`<include>` 标签在这里用于包含其他的 launch 文件，在该文件被启动时，会将其他的 launch 文件一同启动。

这里主要启动了麦克风的初始化、导航、仿真软件的启动文件。

```
<launch>
  <arg name="map" default="" />
  <arg name="master_name" default="$(env MASTER)" />
  <arg name="robot_name" default="$(env HOST)" />
  <arg if="$(eval robot_name == '/')" name="frame_prefix" default="" />
  <arg unless="$(eval robot_name == '/')" name="frame_prefix" default="$(arg robot_name)" />
  <arg name="map_frame" default="$(arg frame_prefix)map" />

  <include file="$(find xf_mic_asr_offline)/launch/mic_init.launch" />
  <include file="$(find hiwonder_navigation)/launch/navigation.launch" />
  <arg name="map" value="$(arg map)" />
  <arg name="master_name" value="$(arg master_name)" />
  <arg name="robot_name" value="$(arg robot_name)" />
</include>

  <node pkg="xf_mic_asr_offline" name="voice_control_navigation" type="voice_control_navigation.py" output="screen" ns="$(arg master_name)" />
  <param name="map_frame" value="$(arg map_frame)" />
  <param name="costmap" value="/$(arg frame_prefix)move_base/local_costmap/costmap" />
</node>

  <include file="$(find hiwonder_navigation)/launch/rviz_navigation.launch" />
</launch>
```

◆ 启动节点

`<node>` 为文件中启动的节点，后续的 `<name>`、`<pkg>`、`<type>`、`<output>` 都为该节点内的参数；

`<name>` 是该节点名字，`<pkg>` 就是该节点所在的功能包；

`<type>` 是该节点执行的文件名称，这里执行的就是

“voice_control_color_sorting.py” 文件的内容；

<output>则是发送的目标，这里的发送目标是屏幕；

<ns> 这个参数可以在自己命名的自定义空间内运行节点。

```
<launch>
  <arg name="map" default="/" />
  <arg name="master_name" default="$(env MASTER)" />
  <arg name="robot_name" default="$(env HOST)" />
  <arg if="$(eval robot_name == '/')" name="frame_prefix" default="/" />
  <arg unless="$(eval robot_name == '/')" name="frame_prefix" default="$(arg robot_name)" />
  <arg name="map_frame" default="$(arg frame_prefix)map" />

  <include file="$(find xf_mic_asr_offline)/launch/mic_init.launch"/>

  <include file="$(find hiwonder_navigation)/launch/navigation.launch">
    <arg name="map" value="$(arg map)" />
    <arg name="master_name" value="$(arg master_name)" />
    <arg name="robot_name" value="$(arg robot_name)" />
  </include>

  <node pkg="xf_mic_asr_offline" name="voice_control_navigation" type="voice_control_navigation.py" output="screen" ns="$(arg master_name)">
    <param name="map_frame" value="$(arg map_frame)" />
    <param name="costmap" value="$(arg frame_prefix)move_base/local_costmap/costmap" />
  </node>

  <include file="$(find hiwonder_navigation)/launch/rviz_navigation.launch"/>
</launch>
```

8.5.2. Python 程序

◆ 订阅服务

订阅获取语音识别词和唤醒角度的服务，连接底层驱动力的节点。

```
31 self.mecanum_pub = rospy.Publisher('hiwonder_controller/cmd_vel', Twist, queue_size=1)
32 self.goal_pub = rospy.Publisher('move_base_simple/goal', PoseStamped, queue_size=1)
33 self.goal_status_pub = rospy.Publisher('move_base/result', MoveBaseActionResult, queue_size=1)
34 rospy.Subscriber('/voice_words', String, self.words_callback)
35 rospy.Subscriber('/mic/awake/angle', Int32, self.angle_callback)
```

◆ 唤醒小车

根据唤醒词，匹配成功后，通过 voice_play 模块播放对应的语音。

```
61 def words_callback(self, msg):
62     self.words = json.dumps(msg.data, ensure_ascii=False)[1:-1]
63     if self.language == 'Chinese':
64         self.words = self.words.replace(' ', '')
65         print('words:', self.words)
66         if self.words is not None and self.words not in ['唤醒成功(wake-up-success)', '休眠(Sleep)', '失败5次(Fail-5-times)',
67                                                         '失败10次(Fail-10-times)']:
68             pass
69         elif self.words == '唤醒成功(wake-up-success)':
70             self.play('awake')
```

◆ 打印声源角度

根据麦克风阵列识别到的声源角度，计算出机器人转向的角度，并打印出来。

```
77 def angle_callback(self, msg):
78     self.angle = msg.data
79     print('angle:', self.angle)
```

◆ 语音控制

匹配识别的语句，通过 voice_play 模块播放对应的语音，再根据识别内容，设定对应

的线速度或角速度，让底层驱动节点 mecanum_pub 发布消息，控制机器人移动。

```
85 while not rospy.is_shutdown() and self.running:
86     if self.words is not None:
87         pose = PoseStamped()
88         pose.header.frame_id = self.map_frame
89         pose.header.stamp = rospy.Time.now()
90         if self.words == '去A点' or self.words == 'go to A point':
91             print('>>>>> go a')
92             pose.pose.position.x = 1
93             pose.pose.position.y = -1
94             pose.pose.orientation.w = 1
95             self.play('go_a')
96             self.goal_pub.publish(pose)
97         elif self.words == '去B点' or self.words == 'go to B point':
98             print('>>>>> go b')
99             pose.pose.position.x = 2
100             pose.pose.position.y = 0
101             pose.pose.orientation.w = 1
102             self.play('go_b')
103             self.goal_pub.publish(pose)
104         elif self.words == '去C点' or self.words == 'go to C point':
105             print('>>>>> go c')
106             pose.pose.position.x = 1
107             pose.pose.position.y = 1
108             pose.pose.orientation.w = 1
109             self.play('go_c')
110             self.goal_pub.publish(pose)
111         elif self.words == '回原点' or self.words == 'go back to the start':
112             print('>>>>> go origin')
113             pose.pose.position.x = 0
114             pose.pose.position.y = 0
115             pose.pose.orientation.w = 1
116             self.play('go_origin')
117             self.goal_pub.publish(pose)
```

8.6. 功能拓展

程序默认的 A 点为机器人起始位置地图的右上角，其对应坐标为（1，-1），单位为米。修改坐标数值即可改变 A 点位置，此处以让 A 点位于起始位置的右下方为例，具体的修改步骤如下：

注意：

1. 用户可以参照本节操作修改 B 点与 C 点位置。
2. 输入指令时需要严格区分大小写，且可使用“Tab”键补齐关键词。

1) 启动机器人，将其连接至远程控制软件 NoMachine, 关于如何连接可参考“1 教程资料\1 机器人快速使用指南\配套手册\1. JetRover 使用入门->4. 开发环境搭建”。

- 2) 点击系统桌面的图标，打开命令行终端。

- 3) 输入指令，并按下回车，进入程序文件目录。

```
roscd xf_mic_asr_offline/scripts/
```

```
hiwonder@hiwonder:~$ roscd xf_mic_asr_offline/scripts/
```

- 4) 输入指令，并按下回车，打开程序文件。

```
vim voice_control_navigation.py
```

```
hiwonder@hiwonder:~/ros_ws/src/xf_mic_asr_offline/scripts$ vim voice_control_navigation.py
```

- 5) 找到下图所示代码：

```
if self.words == '去A点' or self.words == 'go to A point':  
    print('>>>>> go a')  
    pose.pose.position.x = 1  
    pose.pose.position.y = -1  
    pose.pose.orientation.w = 1  
    self.play('go_a')  
    self.goal_pub.publish(pose)
```

- 6) 按下“i”键，进入编辑模式。将“1”修改为“-1”。

```
if self.words == '去A点' or self.words == 'go to A point':  
    print('>>>>> go a')  
    pose.pose.position.x = -1  
    pose.pose.position.y = -1  
    pose.pose.orientation.w = 1  
    self.play('go_a')  
    self.goal_pub.publish(pose)
```

以机器人为第一视角，X轴的正方向为前方，Y轴的正方向为左侧。因此，将A点的X轴坐标由正数修改为负数，即可令该点位于机器人的右下方。

- 7) 修改完成后，按下“Esc”键，输入“:wq”，按下回车，退出并保存文件。

```
:wq
```

- 8) 参照“[3.实验步骤](#)”重新开启玩法，即可查看修改后的玩法效果。

9. 语音控制导航搬运

本节课将语音控制小车，然后在建好的地图上进行导航搬运。

9.1. 实验简介

首先，打开机器人导航服务并加载地图，启动多点导航服务。

然后，订阅麦克风阵列节点发布的语音识别服务，对语音进行声源定位、降噪、识别等操作后，获取识别的语句和声源的角度。

随后，通过麦克风识别语言。识别到唤醒词和控制指令后并且满足设置的识别阈值，机器人会进行相应的语音反馈回答。

最后，根据识别对应的语言，机器人会导航到地图对应的位置，导航会先进行全局规划，行驶中遇到障碍物会进行局部规划。到达第一个点时调用对齐夹取的服务，到达第二个点时调用放置服务。

9.2. 准备工作

1) 在本节课开始前，需要将语音模块安装至机器人上（如已安装可跳过），并连接至集线器的 USB 接口；

2) 请参考本目录下的“[1.六路麦克风阵列配置工作（必读必做！）](#)”参看 Jetson Nano 部分完成对 APPID 的申请和对文件的替换。

3) 系统出厂默认使用英文的唤醒词“hello hiwonder”进行唤醒，如果需要更换语言为中文或使用的是“AI 语音盒子（WonderEchoPro）”需要烧录语音交互应用指令词，参考同目录下“\语音交互应用\中英文唤醒词切换”资料进行语言切换或指令词烧录。

4) 构建机器人当前所处区域的地图。如需了解如何建图，可查看目录“教程资料\ROS1 课程\建图与导航课程”下的文档；

5) 将机器人放置在空旷平台，确保以机器人为中心的周边 3 米内有充足的活动范围。

9.3. 实验步骤

注意：输入指令时需要严格区分大小写，且可使用“Tab”键补齐关键词。

1) 启动机器人，将其连接至远程控制软件 NoMachine。关于如何连接可参考“1 教程资料\1 机器人快速使用指南\配套手册\1.JetRover 使用入门->4.开发环境搭建”。



2) 点击系统桌面的图标，打开命令行终端。

3) 输入指令，并按下回车，关闭手机 APP 服务。

```
sudo systemctl stop start_app_node.service
```

```
> sudo systemctl stop start_app_node.service
```

4) 输入指令，并按下回车，开启语音控制小车玩法：

```
roslaunch xf_mic_asr_offline voice_control_navigation_transport.launch map:=map_01
```

```
└─ roslaunch xf_mic_asr_offline voice_control_navigation_transport.launch map:=map_01
```

指令末端的“map_01”是地图名称，用户可根据需求修改此参数。地图的存放路径是“/home/hiwonder/ros_ws/src/hiwonder_slam/maps”。

9) 如需关闭此玩法，可在终端界面按下“Ctrl+C”。若关闭失败，请反复尝试。

10) 若后续想要重新体验有关手机 APP 的玩法，则需要输入指令重启相关服务。输入并回车执行指令后，等待机械臂恢复至初始姿态即可。

```
sudo systemctl start start_app_node.service
```

```
└─ sudo systemctl start start_app_node.service
```

9.4. 功能实现

启动玩法后，先说唤醒词“hello hiwonder”，再说指令语句，即可控制机器人移动。

例如先说“hello hiwonder”，机器人会回复一声“我在”。接着我们可以发布说“导航搬运”，那么机器人就会对应地去到地图上坐标为（0，0.5，0）的位置夹取物体，夹取完后会前往坐标（1.5，0，0）位置放置。

9.5. 程序分析

9.5.1. launch 启动文件

```
<?xml version="1.0"?>|
<launch>
  <!--
    pick(1.5, 0.5) place(1.0, 1.0)
    |           |
    +-----+
    map
  -->
  <arg name="map"                default="" />
  <arg name="place_without_color" default="true" />
  <arg name="broadcast"          default="true" />
  <arg name="pick_position"      default="[1.5, 0.5, 0, 0, 0]" /> <!--x, y, roll, pitch, yaw-->
  <arg name="place_position"    default="[1.0, 1.0, 0, 0, 0]" /> <!--x, y, roll, pitch, yaw-->

  <include file="$(find xf_mic_asr_offline)/launch/mic_init.launch"/>

  <include file="$(find hiwonder_example)/scripts/navigation_transport/navigation_transport.launch">
    <arg name="map"                value="$(arg map)" />
    <arg name="place_without_color" value="$(arg place_without_color)" />
    <arg name="broadcast"          value="$(arg broadcast)" />
  </include>

  <node pkg="xf_mic_asr_offline" type="voice_control_navigation_transport.py" name="voice_control_navigation_transport_node"
  output="screen">
    <rosparam param="pick_position" subst_value="True">$(arg pick_position)</rosparam>
  </node>
</launch>
```

该 launch 启动文件路径位于：

`/home/hiwonder/ros_ws/src/xf_mic_asr_offline/launch/voice_control_navigation_transport.launch`

◆ 启动多个 launch 文件

`<include>` 标签在这里用于包含其他的 launch 文件，在该文件被启动时，会将其他的 launch 文件一同启动。

这里主要启动了麦克风的初始化、导航的启动文件。通过传递参数控制机器人如何根据预设的位置进行抓取和放置任务。

```
<arg name="map"                default="" />
<arg name="place_without_color" default="true" />
<arg name="broadcast"          default="true" />
<arg name="pick_position"      default="[1.5, 0.5, 0, 0, 0]" /> <!--x, y, roll, pitch, yaw-->
<arg name="place_position"    default="[1.0, 1.0, 0, 0, 0]" /> <!--x, y, roll, pitch, yaw-->

<include file="$(find xf_mic_asr_offline)/launch/mic_init.launch"/>

<include file="$(find hiwonder_example)/scripts/navigation_transport/navigation_transport.launch">
  <arg name="map"                value="$(arg map)" />
  <arg name="place_without_color" value="$(arg place_without_color)" />
  <arg name="broadcast"          value="$(arg broadcast)" />
</include>
```

◆ 启动节点

`<node>` 为文件中启动的节点，后续的 `<name>`、`<pkg>`、`<type>`、`<output>` 都为该节点内的参数；

`<name>` 是该节点名字，`<pkg>` 就是该节点所在的功能包；

`<type>` 是该节点执行的文件名称，这里执行的就是“voice_control_color_sorting.py”文件的内容；

`<output>`则是发送的目标，这里的发送目标是屏幕；

`<ns>` 这个参数可以在自己命名的自定义空间内运行节点。

```
<node pkg="xf_mic_asr_offline" type="voice_control_navigation_transport.py" name="voice_control_navigation_transport_node"
output="screen">
  <rosparam param="pick_position" subst_value="True">$(arg pick_position)</rosparam>
</node>
```

9.5.2. Python 程序

程序源码位于：

/home/hiwonder/ros_ws/src/xf_mic_asr_offline/scripts/voice_control_navigation_transport.py

```
def __init__(self, name):
    rospy.init_node(name)
    self.running = True

    self.language = os.environ['ASR_LANGUAGE']
    self.mic_type = os.environ['MIC_TYPE']
    self.pick_position = rospy.get_param('~pick_position')
    rospy.Subscriber('/asr_node/voice_words', String, self.words_callback)
    rospy.wait_for_service('/voice_control/get_offline_result')
    rospy.wait_for_message("/move_base/local_costmap/costmap", OccupancyGrid)
    self.play('running')
    self.buzzer_pub = rospy.Publisher('/ros_robot_controller/set_buzzer', BuzzerState, queue_size=1)
    rospy.loginfo('唤醒口令: 小幻小幻(Wake up word: hello hiwonder)')
    rospy.loginfo('唤醒后15秒内可以不用再唤醒(No need to wake up within 15 seconds after waking up)')
    rospy.loginfo('控制指令: 导航搬运(Voice command: navigate and transport)')
    try:
        rospy.spin()
    except Exception as e:
        rospy.logerr(str(e))
        rospy.loginfo("Shutting down")
```

◆ 环境变量获取

```
self.language = os.environ['ASR_LANGUAGE']
self.mic_type = os.environ['MIC_TYPE']
```

从系统的环境变量中获取配置项 ASR_LANGUAGE 和 MIC_TYPE，分别代表语音识别语言和麦克风类型。

◆ 创建订阅者

```
rospy.Subscriber('/asr_node/voice_words', String, self.words_callback)
```

监听 /asr_node/voice_words 话题,接收到消息时调用 self.words_callback 方法进行处理。

◆ 创建发布者

```
self.buzzer_pub = rospy.Publisher('/ros_robot_controller/set_buzzer', BuzzerState, queue_size=1)
```

发布到 /ros_robot_controller/set_buzzer 话题，消息类型是 BuzzerState。

◆ 定义回调方法

```
def words_callback(self, msg):
    if self.mic_type == 'xf':
        words = json.dumps(msg.data, ensure_ascii=False)[1:-1]
        if self.language == 'Chinese':
            words = words.replace(' ', '')
    elif self.mic_type == 'WonderEchoPro':
        words = msg.data
    print('words:', words)
    if words is not None and words not in ['唤醒成功(wake-up-success)', '休眠(Sleep)', '失败5次(Fail-5-times)',
                                           '失败10次(Fail-10-times)']:
        if words == '导航搬运' or words == 'navigate and transport':
            pose = Pose2D()
            pose.x = self.pick_position[0]
            pose.y = self.pick_position[1]
            pose.roll = self.pick_position[2]
            pose.pitch = self.pick_position[3]
            pose.yaw = self.pick_position[4]
            res = rospy.ServiceProxy('/navigation_transport/pick', SetPose2D)(pose)
            if res.success:
                self.play('start_navigating')
            else:
                self.play('open_fail')
        elif words == '唤醒成功(wake-up-success)':
            self.play('awake')
        elif words == '休眠(Sleep)':
            msg = BuzzerState()
            msg.freq = 1900
            msg.on_time = 0.05
            msg.off_time = 0.01
            msg.repeat = 1
            self.buzzer_pub.publish(msg)
```

通过接收语音指令并根据指令执行相应的动作，实现导航和物体搬运功能。